DISTRIBUTED SYSTEMS & DATABASE ENGINEERING

How Stripe Scales MongoDB to 5 Million Queries Per Second

Source: Stripe Engineering / Arpit Bhayani Dissection
Architecture: DocDB, Go Proxy Fleet & Data Movement Platform

5M+ QPS	2,000+ Shards	5,000+	99.999%
PEAK TRANSACTIONAL LOAD	PHYSICAL DATABASE NODES	COLLECTIONS MANAGED	FIVE-NINES FINANCIAL SLA

1. The 5M QPS Infrastructure Problem

Stripe processes billions of dollars in global commerce, demanding non-stop transactional execution across petabytes of financial data. Stripe selected MongoDB Community Edition in 2011 for document-schema flexibility and engineering velocity. However, out-of-the-box cluster sharding creates severe operational failure modes when scaled to enterprise dimensions:

Why Out-of-the-Box MongoDB Sharding Breaks at Scale

- **Mongos & Config Replica Set Ceilings:** Native MongoDB sharding routes all traffic through `mongos` routers synchronized by config replica sets. At thousands of shards, metadata contention and lock synchronization create massive latency cliffs.
- **The Hot Shard Nightmare:** Massive enterprise tenants (Shopify, Zoom, Amazon) can suddenly drive 100x traffic surges, saturating individual physical nodes with CPU and disk I/O bottlenecks.
- **Monolithic Shard Recovery:** Physical shards exceeding 1 TB require days to rebuild from backups, increasing Mean Time To Repair (MTTR) during hardware failures.

2. DocDB Architectural Primitives

Stripe resolved this by engineering **DocDB**, a managed internal DBaaS platform built atop MongoDB Community Edition, structured around three core pillars:

Subsystem	Primary Functionality	Scaling Impact
Go Database Proxy Fleet	Stateless routing layer executing auth, rate-limiting, query traffic shaping, and connection multiplexing.	Isolates database instances from client connection storms; shields unindexed queries from crashing shards.
Logical vs Physical Decoupling	Applications query logical chunk keys; the Chunk Metadata Service maps logical chunks to physical nodes.	Enables dynamic online shard migrations without application code modifications or connection drops.
Data Movement Platform	Orchestrates live shard splitting, chunk rebalancing, bin-packing, and automated verification.	Enables zero-downtime, transparent shard rebalancing across thousands of database instances.

3. The Stateless Go Database Proxy Fleet

Every application query traverses a pool of high-performance Go proxies located in front of database shards:

- **Connection Pooling:** Aggregates tens of thousands of application threads into compact, pre-warmed connection pools per mongod node, eliminating thread contention.
- **Dynamic Circuit Breaking:** Monitors query latencies per collection. If a physical shard degrades, the proxy rapidly trips circuit breakers and fails fast, protecting upstream checkout flows.

DISTRIBUTED SYSTEMS & DATABASE ENGINEERING

The Data Movement Platform & Versioned Routing Epochs

4. The Data Movement Platform (DMP) Pipeline

To avoid massive monolithic shards, DocDB maintains small, fine-grained shards and rebalances data dynamically. The Data Movement Platform executes live migrations across five seamless stages:

5-Stage Live Shard Migration Orchestration

1. **Intent Registration & Provisioning:** The migration intent is registered in the control plane coordinator. Target collections and secondary indexes are pre-built on the target shard.
2. **Cursor-Based Bulk Copy:** A consistent point-in-time snapshot point (T_0) is established. Offline parallel workers stream data using cursor-based pagination from source to target.
3. **Continuous CDC Catch-Up:** Change Data Capture (CDC) streams capture all ongoing mutations applied to the source shard after T_0, continuously replicating updates to the target.
4. **Correctness & Parity Auditing:** Automated background verification jobs audit record counts and checksums across source and target until 100% convergence is confirmed.
5. **Versioned Config-Flip:** The migration coordinator updates routing epochs, cutting over live traffic with zero application downtime.

5. The Versioned Config-Flip Mechanism

In a distributed proxy fleet spanning hundreds of nodes, atomically broadcasting a routing update to all proxies simultaneously is impossible. A distributed race condition occurs where some proxies write to the legacy shard while others write to the new shard, causing catastrophic split-brain mutations.

The Versioned Epoch Solution

Stripe resolves this through **Versioned Routing Epochs**:

- Every chunk mapping possesses a monotonically increasing integer epoch: $E_{ ext{chunk}}$.
- During cutover, the target shard is updated with epoch $E_{ ext{new}} = E_{ ext{old}} + 1$, and the source shard is marked read-only.
- Proxies stamp every outbound query with their locally cached epoch version.
- If a proxy with a stale epoch attempts to write to the old shard, the shard immediately rejects the query with a `STALE_ROUTING_EPOCH` exception.
- The proxy catches the rejection, invalidates its local cache, queries the Chunk Metadata Service for the latest epoch, and seamlessly retries against the new shard in milliseconds.

6. Dynamic Shard Splitting for Hot Tenants

When high-volume merchants generate sudden transaction surges (e.g., flash sales, Black Friday), their specific logical chunk is split from shared multi-tenant hardware and migrated to dedicated physical instances in minutes with zero customer disruption.

DISTRIBUTED SYSTEMS & DATABASE ENGINEERING

Elastic Bin-Packing, Trade-offs & Engineering Takeaways

7. Elastic Bin-Packing & Infrastructure Cost Optimization

While shard splitting addresses traffic spikes, leaving thousands of over-provisioned shards during normal traffic creates massive cloud waste. DocDB uses **Bin-Packing Algorithms** to consolidate underutilized chunks:

Traffic Surge Phase
Surging merchants are dynamically isolated onto dedicated physical hardware. Shards are kept below 50% CPU and IOPS thresholds to absorb unexpected flash sales.

Consolidation Phase
During quiet periods, the Data Movement Platform aggregates hundreds of cold logical chunks onto shared physical instances, reclaiming hundreds of database nodes.

8. Critical Architectural Trade-offs

Design Decision	Primary Advantages	Engineering Trade-offs & Costs
Custom Proxies vs Native Drivers	Connection pooling, rate limiting, and transparent live migration without client SDK updates.	Introduces a network hop (~0.5ms latency) and requires maintaining a dedicated Go proxy codebase.
Fine-Grained Sharding	Ultra-fast MTTR, swift replica resyncs, and localized blast radius during node failures.	Higher metadata coordination overhead across the central Chunk Metadata Service.
Lazy Epoch-Based Invalidation	Eliminates distributed 2-Phase Commit (2PC) locks across thousands of proxies.	Requires application retries to handle brief `STALE_ROUTING_EPOCH` exceptions during cutovers.

9. Core Engineering Principles for High-Throughput Databases

Takeaways for Engineering Leaders

1. **Keep Shards Granular and Small:** Shards measured in hundreds of gigabytes—not tens of terabytes—can be backed up, restored, and migrated effortlessly.
2. **Decouple Application Routing from Hardware:** Proxies and logical chunk abstractions give infrastructure teams operational freedom to balance clusters without product friction.
3. **Prefer Lazy Convergence Over Distributed Locks:** Versioned epochs turn complex distributed cutovers into simple, self-healing, rejection-based cache invalidations.